



INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Ingeniería Informática

PROYECTO ESPECIAL T.L.A

Diseño e Implementación de un Lenguaje: BoardSim

Stage III: Backend & Informe Final

Grupo G-96 (BoardSim)

Arias, Nicolás Valentín (62272)

Dubovitsky, Keoni Lucas (62815)

Ferrari, Franco (63094)

Lasserre, Facundo (62165)

Índice

1. Introducción	3
2. Modelo Computacional	3
2.1. Dominio	3
2.2. Lenguaje	3
3. Implementación	3
3.1. Frontend	4
3.2. Backend	4
3.2.1. Análisis Semántico	4
3.2.2. Motor de Simulación	4
3.3. Adicionales: Implementación de Código Relevante	4
3.4. Dificultades Encontradas	5
3.4.1. Eliminación de “Doble Parsing”	5
3.4.2. Gestión de Memoria	5
4. Futuras Extensiones	5
5. Conclusiones	5
6. Bibliografía	6

1. Introducción

El proyecto **BoardSim** consiste en el diseño e implementación de un Lenguaje de Dominio Específico (DSL) orientado a la simulación y prototipado rápido de juegos de mesa. El objetivo principal es abstraer la complejidad técnica inherente a la programación de simulaciones (manejo de memoria, bucles de juego, azar) para permitir a los diseñadores definir reglas y entidades de manera intuitiva.

En esta etapa final, se completa el desarrollo mediante la integración de un **Backend** funcional. A diferencia de compiladores tradicionales, BoardSim utiliza una arquitectura de interpretación sobre un Árbol de Sintaxis Abstracta (AST) enriquecido semánticamente, permitiendo validaciones estrictas y ejecución dinámica de la lógica del juego.

2. Modelo Computacional

2.1. Dominio

El dominio modelado abarca juegos de mesa basados en turnos y gestión de recursos. Las entidades fundamentales abstraídas son:

- **Tablero (Board):** Soporta topologías *Loop* (cíclicas) y *Graph* (grafos interconectados).
- **Celdas (Cells):** Unidades con propiedades semánticas como costo, renta o eventos.
- **Jugadores (Players):** Agentes con estado mutable (dinero, posición) y estrategias (*aggressive*, *random*).
- **Azar (Dice):** Generadores de entropía configurables.

2.2. Lenguaje

BoardSim es un lenguaje de tipado estático fuerte que combina dos paradigmas:

1. **Declarativo:** Para la configuración inicial del entorno (tablero, celdas, jugadores).
2. **Imperativo:** Un bloque `simulate` que orquesta el flujo mediante sentencias de control (`if`, `while`) y manipulación de variables.

3. Implementación

La arquitectura es modular, dividiendo responsabilidades entre Frontend (análisis) y Backend (ejecución), comunicados mediante un estado global (**CompilerState**).

3.1. Frontend

Implementado con **Flex** y **Bison**.

- **Análisis Léxico:** Usa *Push Parsing* reentrante y Contextos de Flex para el manejo robusto de cadenas, eliminando *backtracking* y copias manuales de memoria.
- **Análisis Sintáctico:** Construye un AST jerárquico que preserva la estructura semántica de las operaciones para evitar la degradación a texto plano.

3.2. Backend

Instancia un motor de simulación en C.

3.2.1. Análisis Semántico

Valida reglas de negocio previas a la ejecución: unicidad de tableros, límites de jugadores (máx. 8) y coherencia referencial de celdas.

3.2.2. Motor de Simulación

Transforma el AST en un estado dinámico (**SimulationState**). Incluye una heurística avanzada para inferir el tipo de juego analizando tanto la topología del tablero como la semántica de los identificadores utilizados.

3.3. Adicionales: Implementación de Código Relevante

A continuación, se presentan fragmentos clave del código fuente que demuestran la resolución de los desafíos técnicos planteados.

1. Heurística de Inferencia de Reglas (BoardSim.c) El backend implementa un sistema de inferencia que analiza la **topología del tablero** y la **semántica de los nombres** para determinar automáticamente el conjunto de reglas a aplicar.

```
1 GameType detectGameType(GameConfig config) {
2     // Grafos: tamaño 0 (celdas dinamicas)
3     if (config.boardSize == 0) return GAME_TYPE_GRAPH;
4
5     // Ajedrez: Topologia 8x8 y semantica de piezas (king, queen...)
6     if (hasChessNames(config)) return GAME_TYPE_CHESS;
7
8     // Monopoly: Tableros medianos con dados y propiedades
9     if (config.boardSize >= 6 && config.diceCount > 0) {
10         // Analisis semantico de nombres (bank, avenue, tax...)
11         if (hasPropertyName(config)) return GAME_TYPE_MONOPOLY;
12         return GAME_TYPE_MONOPOLY;
13     }
14     return GAME_TYPE_GENERIC;
15 }
```

Listing 1: Inferencia de reglas basada en contexto

2. Estructuras Semánticas para evitar “Doble Parsing” (BisonActions.c)

Para corregir el error de almacenar condiciones como texto, se implementó la construcción de nodos semánticos tipados directamente en el parser.

```
1 Condition* ComparisonExpressionSemanticAction(char* left, int right,
2   TokenLabel operator) {
3   ComparisonOperator op;
4   switch (operator) {
5       case GREATER_THAN: op = CMP_GREATER_THAN; break;
6       case LESS_THAN:    op = CMP_LESS_THAN; break;
7       // ... otros casos
8   }
9   // Se crea la estructura semantica, NO un string formateado
10  ComparisonExpression* expr = createComparisonExpression(left, right,
11  op);
12  return createComparisonCondition(expr);
13 }
```

Listing 2: Construcción semántica de comparaciones

3.4. Dificultades Encontradas

3.4.1. Eliminación de “Doble Parsing”

Problema: Las condiciones lógicas se almacenaban como texto en el AST, obligando a re-parsearlas en runtime. *Solución:* Se refactorizó el AST para usar nodos `ComparisonExpression` con operadores `enum`, permitiendo evaluación directa y eficiente.

3.4.2. Gestión de Memoria

Problema: Fugas de memoria por manejo manual de cadenas en el Lexer. *Solución:* Se implementaron destructores recursivos (`destroyASTNode`) y se delegó la propiedad de las cadenas al AST, limpiando la lógica de Flex mediante el uso de buffers dinámicos y contextos.

4. Futuras Extensiones

- **IA Avanzada:** Permitir scripting de estrategias personalizadas en el bloque `simulate`.
- **Visualización:** Exportación del estado del tablero a formato DOT/Graphviz.
- **Multijugador:** Soporte para sockets que permitan entradas externas.

5. Conclusiones

El desarrollo de BoardSim ha logrado integrar exitosamente los conceptos de la cátedra en una aplicación robusta. La arquitectura final demuestra una clara separación de responsabilidades y la capacidad de adaptar reglas de simulación dinámicamente aporta un gran valor a la experiencia de desarrollo.

Asimismo, se han seguido los lineamientos y especificaciones detallados en la documentación provista por la cátedra, listada a continuación.

6. Bibliografía

- (2025-08-21, v3.0.8) *Proyecto Especial*. Documento de Cátedra.
- (2025-03-13, v0.1.0) *Modelo Computacional*. Documento de Cátedra.
- (2025-09-30, v1.0.1) *Especificación*. Documento de Cátedra.
- (2025-09-03, v1.0.0) *Análisis Léxico*. Documento de Cátedra.
- (2024-05-08, v0.1.0) *Análisis Sintáctico*. Documento de Cátedra.
- (2024-09-25, v0.2.0) *Análisis Semántico*. Documento de Cátedra.
- (2024-05-16, v0.1.0) *Runtime*. Documento de Cátedra.
- (2024-05-16, v0.1.0) *Generación de Código*. Documento de Cátedra.
- (2025-11-26, v0.1.0) *Corrección del Frontend*. Feedback de Cátedra.
- Levine, J. R. (2009). *flex & bison*. O'Reilly Media, Inc.